

โครงงาน

เรื่อง ระบบจัดการร้านกาแฟ (Cafe Ordering System)

กลุ่มที่ 16 (BarisTech)

รหัสนิสิต	ชื่อ-นามสกุล
67160026	นางสาวฑิมพิกา ภุมรินทร์
67160046	นางสาวอภิชญา กันประดับ
67160389	นางสาวจิดาภา สิทธะนะ
67160401	นางสาวปภาวรินทร์ ทัศนางกูร
67160408	นางสาวยุพากร สอนเถื่อน

เสนอ

อาจารย์ วิทวัส พันธุมจินดา

โครงงานนี้เป็นส่วนหนึ่งของการศึกษาตามรายวิชา 88734065

System Analysis and Designs (การวิเคราะห์และออกแบบระบบ)ภาคเรียนที่ 1 ปีการศึกษา 2569

สารบัญ

1. บทนำ 4

1.1 บริบทและปัญหา 4

1.2 วัตถุประสงค์ของโครงงาน 4

1.3 องค์ประกอบของระบบ (Team Charter, wk01) 4

1.4 แบบจำลองการพัฒนา 4

2. การวิเคราะห์ความเป็นไปได้ 5

2.1 BPMN กระบวนการเดิม (As-Is) 6

2.2 BPMN กระบวนการที่ต้องการ (To-Be) 7

3. การวิเคราะห์ความต้องการ 8

3.1 สรุปการสัมภาษณ์ 8

3.2 ความต้องการเชิงหน้าที่ (Functional Requirements) 8

3.3 ความต้องการที่ไม่ใช่เชิงหน้าที่ และข้อจำกัด 8

3.4 การปรับ requirement ระหว่างโครงการ (สัปดาห์ 13) 9

4. User Stories และ Use Case 10

4.1 Use Case Diagram 10

4.2 Use Case: รับออเดอร์และคำนวณราคา 10

4.3 User Story และ Acceptance Criteria 11

5. การออกแบบเชิงวัตถุ (OOAD) 12

5.1 Class Diagram 12

5.2 Object Diagram 13

5.3 ความสอดคล้องระหว่าง diagram กับโค้ดจริง 13

6. การออกแบบฐานข้อมูล 14

6.1 ER Diagram (ฉบับปัจจุบัน) 14

6.2 ตารางและความสัมพันธ์ 14

6.3 การปรับปรุงหลังสัปดาห์ 7 (ER Diagram ฉบับสัปดาห์ 7 มี 10 ตาราง) 15

7. การออกแบบกระบวนการและพฤติกรรมระบบ 16

7.1 Sequence Diagram: สร้างออเดอร์ (POST /api/orders) 16

7.2 Business Rules 16

7.3 ความสอดคล้องกับโค้ด 17

8. การออกแบบ Input/Output และ UI 18
8.1 Wireframe (Mid-Fi) 18
8.2 หน้าจอที่ implement จริง 19
9. สถาปัตยกรรมระบบ 22
9.1 4+1 View 22
9.2 การแบ่ง component 23
10. ผลการทดสอบ Usability 24
11. ผลการรวมระบบ (Integration) 25
11.1 ผลทดสอบ End-to-End 25
12. ผลการตรวจสอบคุณภาพและความปลอดภัย 26
12.1 ความเสี่ยงที่พบ 26
12.2 การแก้ไขความเสี่ยงสูงสุด 26
12.3 ผลทดสอบหลังแก้ 26
13. สรุปผลและข้อเสนอแนะ 27
13.1 สิ่งที่ทำได้ 27
13.2 บทเรียน 27
13.3 ข้อจำกัดของระบบ 27
13.4 แนวทางพัฒนาต่อ 27
ภาคผนวก 28
ภาคผนวก ก: AI Disclosure Form ที่ใช้ตลอดเทอม 28
ภาคผนวก ข: เอกสารที่เก็บแยกไฟล์ 28

1.บทนำ

1.1 บริบทและปัญหา

ร้านกาแฟที่มี 3–5 สาขาใช้วิธีจดออเดอร์ด้วยกระดาษและคิดเงินด้วยเครื่องคิดเลข ในช่วงที่ลูกค้าเยอะ พนักงานจดไม่ทัน ออเดอร์ตกหล่น เลขโต๊ะสลับกัน เจ้าของร้านควบคุมสต็อกวัตถุดิบไม่ได้ และไม่มีสรุปยอดขายรายวัน รายเดือน หรือรายสาขาไปใช้ตัดสินใจเรื่องเมนู (จากการสัมภาษณ์จำลองใน wk03)

1.2 วัตถุประสงค์ของโครงงาน

- พัฒนาระบบ POS สำหรับรับออเดอร์ คิดเงิน ทอนเงิน และออกใบเสร็จ เพื่อลดความล่าช้าและข้อผิดพลาดจากการจดมือ
- ตัดสต็อกวัตถุดิบอัตโนมัติตามสูตรของแต่ละเมนู และแจ้งเตือนเมื่อวัตถุดิบใกล้หมด
- สรุปรายงานยอดขายรายวัน รายเดือน แยกตามสาขา วิธีชำระเงิน และเมนู
- แยกสิทธิ์ผู้ใช้ตามบทบาท (เจ้าของร้าน / แคชเชียร์) และผ่านการตรวจสอบคุณภาพและความปลอดภัย

1.3 องค์ประกอบของระบบ (Team Charter, wk01)

องค์ประกอบ	รายละเอียด
People	พนักงานหน้าร้าน, บาริสต้า, เจ้าของร้าน/ผู้จัดการ, พนักงานบัญชี, ฝ่ายจัดซื้อ
Process	รับออเดอร์, ชำระเงิน, ตัดสต็อกวัตถุดิบ (สมัครสมาชิก/สะสมแต้มอยู่นอกขอบเขตที่พัฒนา)
Data	รายการเมนู ราคา จำนวนสต็อกวัตถุดิบ ประวัติการสั่งซื้อ
Hardware	เครื่อง POS, แท็บเล็ตรับออเดอร์, เครื่องพิมพ์ใบเสร็จ, จอฝั่งลูกค้า, เครื่องสแกน QR
Software	แอปพลิเคชัน POS, ระบบจัดการสต็อก, ระบบรายงานยอดขาย
Network	Wi-Fi ในร้านเชื่อมเครื่อง POS กับเซิร์ฟเวอร์ฐานข้อมูล

1.4 แบบจำลองการพัฒนา

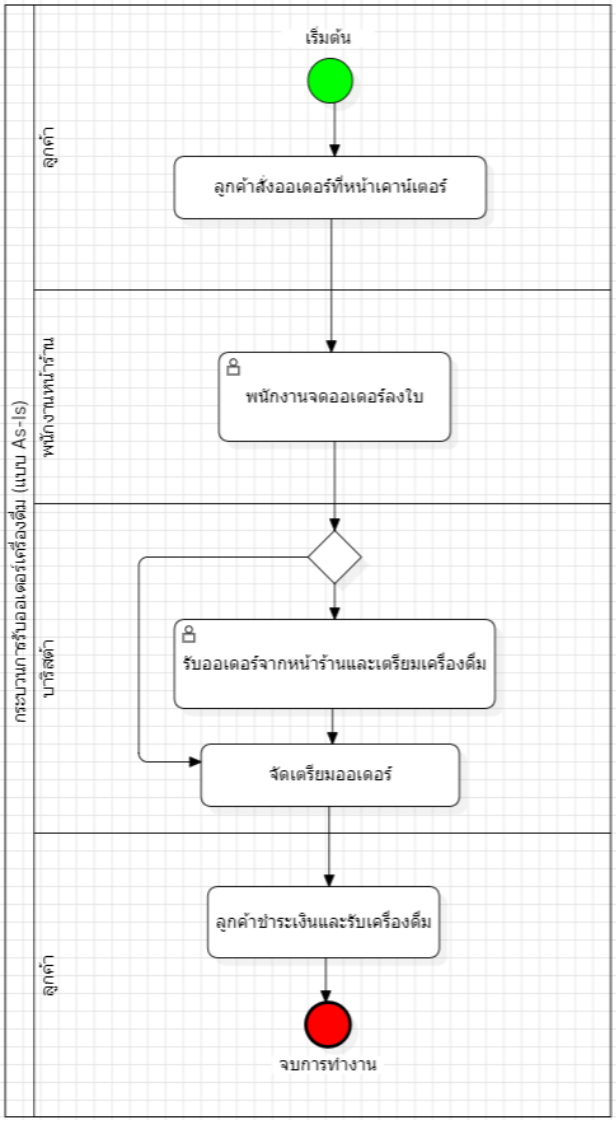
เลือก **Agile** เพราะต้องการความยืดหยุ่นและรับ feedback ต่อเนื่อง รับมือ requirement ที่ยังไม่นิ่งได้ และเหมาะกับทีมเล็ก (5 คน) ที่ส่งงานทุกสัปดาห์ งานทุกสัปดาห์ต่อยอดกันเป็นเรื่องเดียว คือ BPMN (wk2) → Requirements (wk3) → User Story (wk4) → Class/ER/Sequence (wk6–9) → UI (wk10) → Architecture (wk11) → Usability (wk12) → Integration (wk13) → Quality (wk14)

2.การวิเคราะห์ความเป็นไปได้

รายงานความเป็นไปได้ฉบับย่อ (wk02) สรุปว่า **ควรดำเนินโครงการต่อ** เพราะผลประโยชน์คุ้มค่าต้นทุน และทีมพร้อมทั้งด้านเทคนิคและเวลา

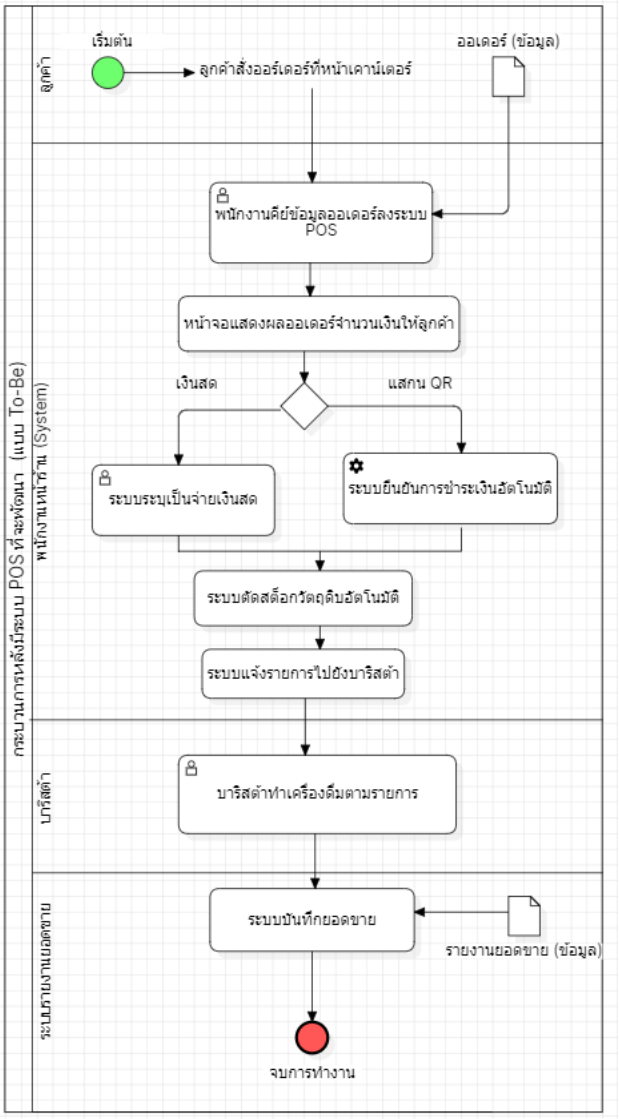
ด้าน	ข้อสรุป
Technical	ทีมมีทักษะ Node.js/Express และ MySQL ระดับปานกลาง ระบบรองรับอุปกรณ์หน้าร้านทั่วไป (เครื่อง POS, แท็บเล็ต, เครื่องพิมพ์ใบเสร็จ)
Economic	ต้นทุนพัฒนาและอุปกรณ์ประมาณ 60,000–80,000 บาท ลดขั้นตอนรับออเดอร์/คิดเงินด้วยมือ ลดข้อผิดพลาด และได้ข้อมูลยอดขายไปใช้ต่อ
Operational	พนักงานที่คุ้นกับกระดาษต้องปรับตัว จึงเน้นออกแบบหน้าจอให้ใช้ง่ายและมีการอบรม 1–2 วัน
Schedule	16 สัปดาห์: สัปดาห์ 1–4 วิเคราะห์/ออกแบบ, 5–12 พัฒนา, 13–16 ทดสอบและนำเสนอ

2.1 BPMN กระบวนการเดิม (As-Is)



รูปที่ 2.1 BPMN แบบ As-Is: จดออเดอร์ด้วยมือ

2.2 BPMN กระบวนการที่ต้องการ (To-Be)



รูปที่ 2.2 BPMN แบบ To-Be: ใช้ระบบ POS

To-Be แก้ปัญหา Operational โดยลดความสับสนและออเดอร์ตกหล่นจากการจดมือส่งออเดอร์ให้บาริสต้าผ่านระบบ และแก้ปัญหา Economic โดยแลกต้นทุนระบบกับเวลาที่ประหยัดและข้อผิดพลาดที่ลดลง

3. การวิเคราะห์ความต้องการ

3.1 สรุปการสัมภาษณ์

สัมภาษณ์จำลอง (สมาชิก 1 คนสวมบทเจ้าของร้าน) ด้วยคำถาม 8 ข้อ สรุปได้ว่า

ประเด็น	สิ่งที่ได้จากการสัมภาษณ์
ปัญหาปัจจุบัน	จดออเดอร์ด้วยกระดาษ คิดเงินด้วยเครื่องคิดเลข ช่วงลูกค้าเยอะจดไม่ทัน ออเดอร์ตกหล่น เลขโต๊ะสลับ
สิ่งที่ต้องการจากระบบ	ช่วยจัดออเดอร์ ช่วยเรื่องสต็อกจะได้ไม่ต้องจดเอง
รายงานที่ต้องการ	สรุปยอดรายวัน รายเดือน รายสาขา เพื่อดูว่าเมนูไหนควรเอาออกหรือแนะนำ
งบประมาณ / เวลา	60,000–80,000 บาท (รวมพัฒนา อุปกรณ์ อบรม ติดตั้ง) · เสร็จใน 3–4 เดือน ยอมรับ 16 สัปดาห์ถ้ามีการอัปเดตและส่ง demo ให้ทดลองสม่ำเสมอ
การปรับตัวของพนักงาน	พนักงานเรียนรู้ได้ ต้องการอบรมประมาณ 1–2 วัน

3.2 ความต้องการเชิงหน้าที่ (Functional Requirements)

รหัส	ความต้องการ
FR-01	บันทึกออเดอร์พร้อมเลขโต๊ะหรือประเภทการทาน
FR-02	คำนวณยอดรวม เงินทอน และแสดงยอดชำระ
FR-03	ส่งออเดอร์และแสดงลำดับคิวที่หน้าจอบาริสต้า
FR-04	จัดการวัตถุดิบและตัดสต็อกอัตโนมัติหลังชำระเงิน
FR-05	แสดงจำนวนคงเหลือของวัตถุดิบ
FR-06	ออกใบเสร็จรับเงิน
FR-07	สรุปรายงานยอดขายรายวัน รายเดือน และแยกตามสาขา
FR-08	แสดงอันดับเมนูยอดนิยม
FR-09 (เพิ่ม wk13)	แสดงจำนวนที่ขายได้ และเตือนสต็อกตั้งแต่ตอนเลือกสินค้า
FR-10 (เพิ่ม wk13)	แคชเชียร์ดูประวัติออเดอร์ พิมพ์ใบเสร็จซ้ำ และยกเลิกออเดอร์ที่ยังไม่เริ่มทำ
FR-11 (เพิ่ม wk13)	สรุปยอดขายรวมต่อเมนู ตามสาขาและช่วงเวลา

3.3 ความต้องการที่ไม่ใช่เชิงหน้าที่ และข้อจำกัด

รหัส	ความต้องการ
NFR-01	คำนวณยอด ออกใบเสร็จ และอัปเดตสต็อกภายใน 2 วินาทีหลังยืนยันการชำระเงิน
NFR-02	ส่งข้อมูลออเดอร์ไปบาริสต้าภายใน 2 วินาที
NFR-03	หน้าจอใช้งานง่าย เรียนรู้ขั้นพื้นฐานได้ภายใน 1–2 วัน

รหัส	ความต้องการ
NFR-04	กำหนดสิทธิ์การเข้าถึงข้อมูลตามบทบาท
NFR-05	อัปเดตเมนู ราคา วัตถุดิบ โดยไม่กระทบการรับออเดอร์หน้าร้าน
NFR-06 (เพิ่ม wk13)	องค์ประกอบสำคัญของการชำระเงินต้องเด่นชัด ผู้ใช้ใหม่หาเจอเอง
ข้อจำกัด	งบ 60,000–80,000 บาท · เสร็จภายใน 3–4 เดือน (ประมาณ 16 สัปดาห์) · ไม่ทำระบบบัญชีภาษีเต็มรูปแบบ

3.4 การปรับ requirement ระหว่างโครงการ (สัปดาห์ 13)

หลังทดสอบ usability และรวมระบบ พบว่า requirement เดิมไม่ครอบคลุมสถานการณ์จริง จึงเพิ่ม FR-09, FR-10, FR-11 และ NFR-06 พร้อมบันทึกที่มาไว้ท้ายไฟล์ wk03-requirements.md เช่น FR-04 เดิมกำหนดแค่การตัดสต็อกหลังชำระเงิน แต่ผู้ทดสอบต้องการให้เตือนตั้งแต่ตอนเลือกสินค้า (FR-09)

4. User Stories และ Use Case

4.1 Use Case Diagram



รูปที่ 4.1 Use Case Diagram ระบบจัดการร้านกาแฟ (ผู้ใช้: พนักงาน, บาริสต้า, เจ้าของร้าน)

4.2 Use Case: รับออเดอร์และคำนวณราคา

Actor: พนักงาน · **Precondition:** เข้าสู่ระบบแล้วและระบบพร้อมรับออเดอร์

Main Flow: (1) เลือกเมนู (2) ระบบแสดงรายการและราคา (3) เลือกทานที่ร้าน/กลับบ้าน (4) ทานที่ร้านระบุเลขโต๊ะ (5) ระบบคำนวณยอดรวม (6) เลือกวิธีชำระ (7) รับเงิน (8) ระบบตรวจจำนวนเงิน (9) คำนวณเงินทอน (10) บันทึกออเดอร์และสถานะชำระเงิน (11) ส่งข้อมูลให้บาริสต้า (12) ออกใบเสร็จ

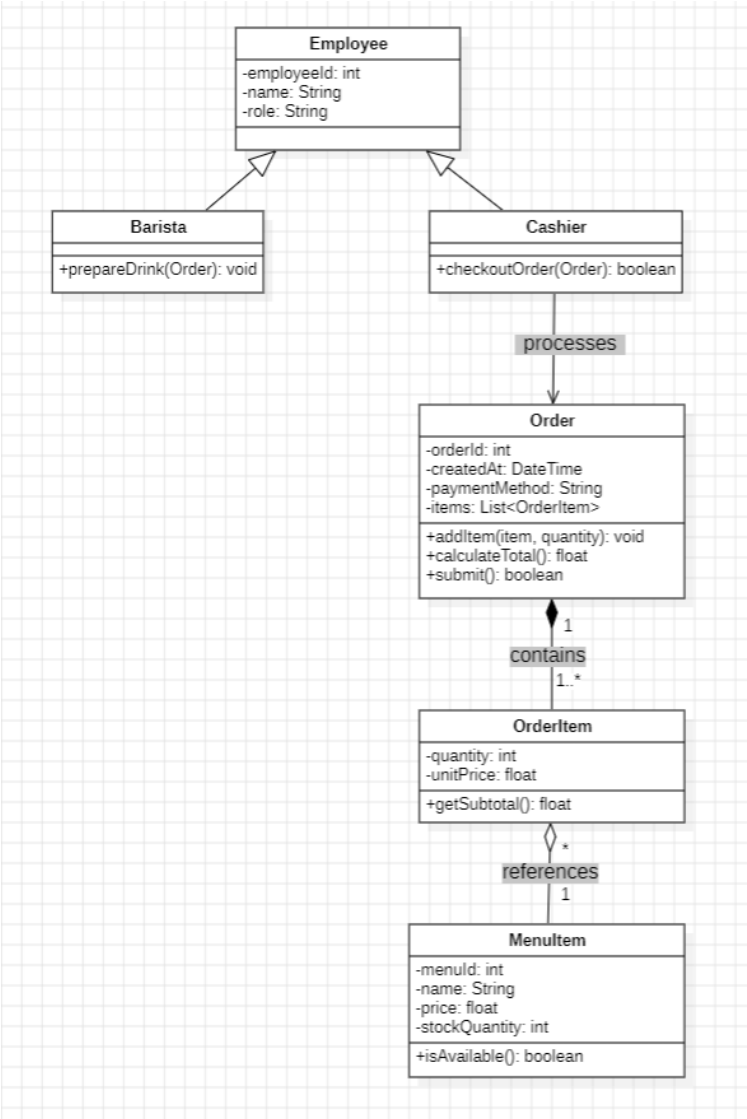
Alternative Flow: 3a กลับบ้านไม่ต้องระบุเลขโต๊ะ · 5a ใช้ส่วนลดแล้วคำนวณยอดใหม่ · **Exception Flow:** 8a จำนวนเงินไม่เพียงพอ ระบบแจ้ง "จำนวนเงินไม่เพียงพอ" และไม่บันทึก

4.3 User Story และ Acceptance Criteria

User Story	Acceptance Criteria หลัก
US-01 รับออเดอร์และคำนวณราคา	ต้องมีรายการสินค้า · ราคาและจำนวนต้องมากกว่า 0 · ยอดรวม = ราคา × จำนวน
US-02 ระบุประเภทการทาน	ทานที่ร้านต้องระบุเลขโต๊ะ · กลับบ้านไม่ต้อง
US-03 ใช้ส่วนลด	ยอดชำระ = ยอดรวม − ส่วนลด
US-04 ชำระเงินและเงินทอน	เงินไม่พอแจ้ง "จำนวนเงินไม่เพียงพอ" · เงินทอน = เงินที่รับ − ยอดชำระ
US-05 ตรวจสอบวัตถุดิบ	วัตถุดิบพอจึงยืนยันได้ · ไม่พอแจ้ง "วัตถุดิบไม่เพียงพอ" และไม่ยืนยัน
US-06 บันทึกและดำเนินการออเดอร์	บันทึกสถานะชำระเงิน · ส่งข้อมูลให้บาริสต้า · ออกใบเสร็จ

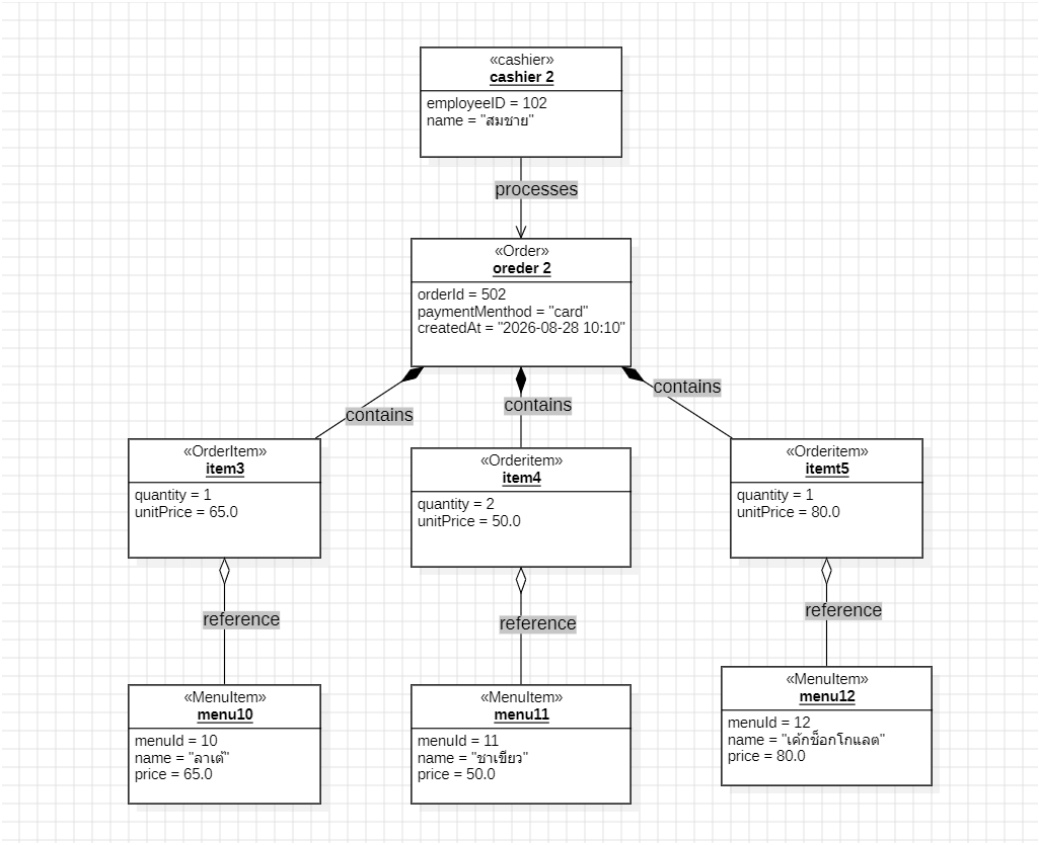
5. การออกแบบเชิงวัตถุ (OOAD)

5.1 Class Diagram



รูปที่ 5.1 Class Diagram (wk06): Employee/Barista/Cashier, Order, OrderItem, MenuItem

5.2 Object Diagram



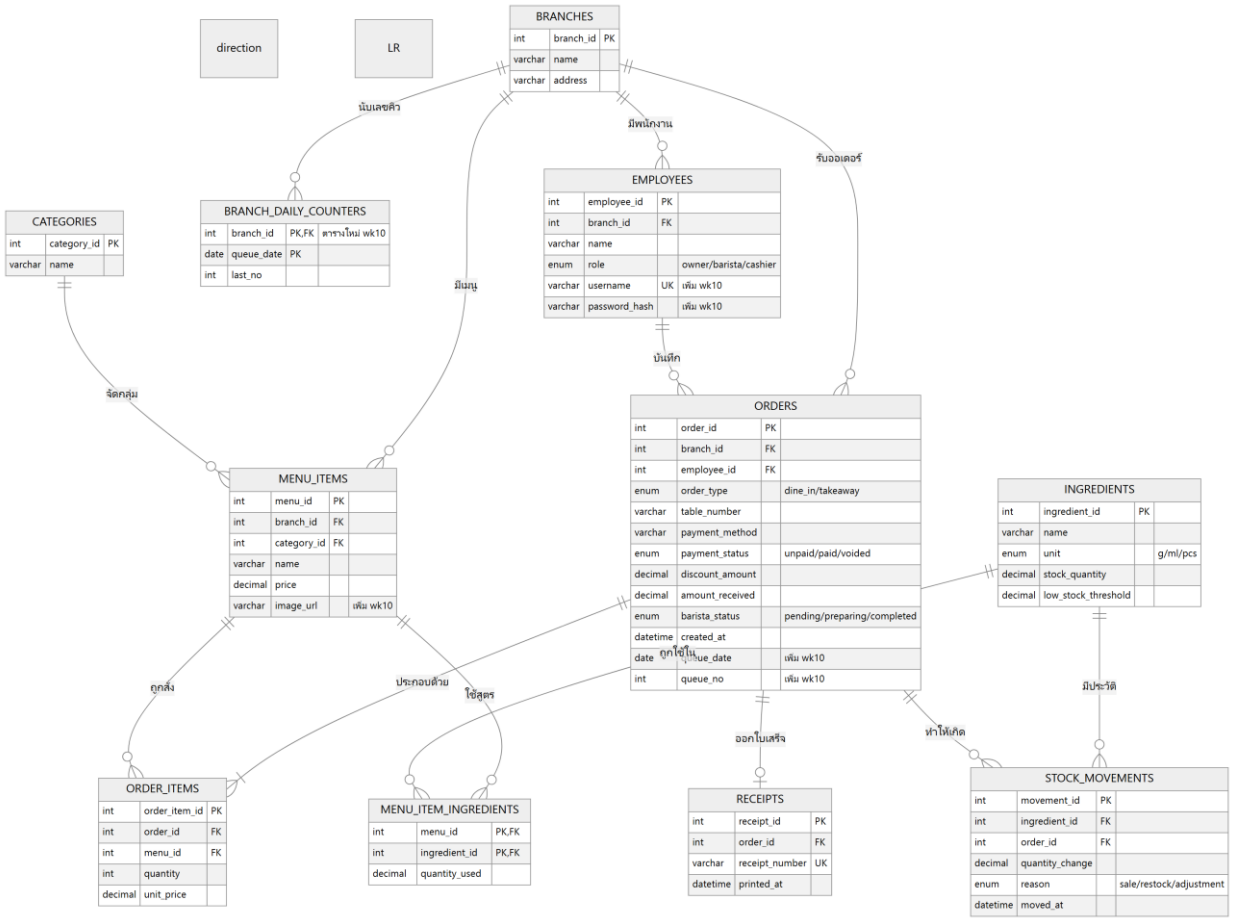
รูปที่ 5.2 Object Diagram (wk06): ตัวอย่างอ็อบเจ็กต์ของออเดอร์

5.3 ความสอดคล้องระหว่าง diagram กับโค้ดจริง

ใน **Class Diagram**	ในโค้ดจริง	หมายเหตุ
Order (addItem, calculateTotal, submit)	code/src/models/Order.js คลาส Order	ตรงตาม diagram ใช้ใน orderController.createOrder
OrderItem (quantity, unitPrice, getSubtotal)	คลาส OrderItem ในไฟล์เดียวกัน	ตรง unitPrice เก็บราคา ณ ตอนสั่ง (snapshot)
MenuItem (price, stockQuantity)	ตาราง menu_items + menuModel.js	ต่างจาก diagram: สต็อกไม่ได้อยู่ที่เมนู แต่อยู่ที่วัตถุดิบ (ingredients) ตามสูตร menu_item_ingredients
Employee, Barista, Cashier (สืบทอด)	ตาราง employees คอลัมน์ role + middleware auth.js	ต่างจาก diagram: แยกสิทธิ์ด้วยบทบาทในข้อมูล ไม่ได้ทำเป็นคลาสสืบทอด
Cashier.checkoutOrder	orderController.createOrder	เทียบเท่าการทำงาน
Barista.prepareDrink	ยังไม่ได้ implement	ยังไม่มีหน้าจอบาริสต้า (FR-03) มีเฉพาะสถานะ barista_status

6. การออกแบบฐานข้อมูล

6.1 ER Diagram (ฉบับปัจจุบัน)



รูปที่ 6.1 ER Diagram ฉบับปัจจุบัน 11 ตาราง ตรงกับ code/schema.sql (ข้อความ "เพิ่ม wk10" คือส่วนที่เพิ่มหลังสัปดาห์ 7)

6.2 ตารางและความสัมพันธ์

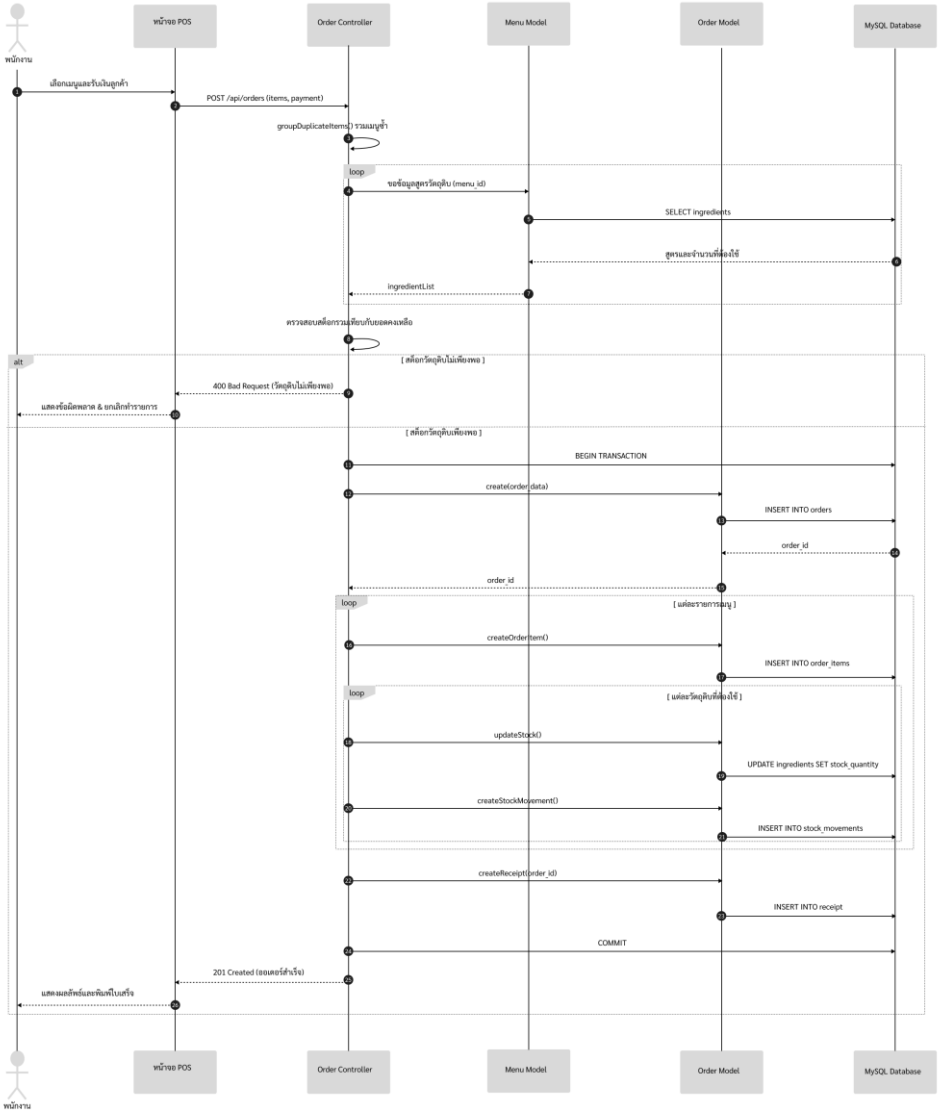
ตาราง	หน้าที่
branches, employees	สาขา และพนักงานแต่ละสาขา (role: owner / barista / cashier)
categories, menu_items	หมวดหมู่ และเมนูของแต่ละสาขา (ราคา รูป)
ingredients, menu_item_ingredients	วัตถุดิบ (ใช้ร่วมกันทุกสาขา) และสูตรว่าเมนูหนึ่งใช้วัตถุดิบอะไรเท่าไร
orders, order_items	ออเดอร์ และรายการสินค้าในออเดอร์ (เก็บราคา ณ ตอนสั่ง)
stock_movements	ประวัติเพิ่ม/ลดสต็อก (sale, restock, adjustment)
receipts	ใบเสร็จของออเดอร์
branch_daily_counters	เลขคิวล่าสุดของแต่ละสาขาในแต่ละวัน

6.3 การปรับปรุงหลังสัปดาห์ 7 (ER Diagram ฉบับสัปดาห์ 7 มี 10 ตาราง)

- สัปดาห์ 9: เพิ่ม menu_items.branch_id (กันการเข้าถึงเมนูข้ามสาขา), เพิ่มสถานะ voided ใน orders.payment_status (ยกเลิกบิลโดยไม่ลบข้อมูล), เอาคอลัมน์ที่คำนวณได้ (ยอดรวม ยอดสุทธิ เงินทอน) ออกจาก orders ตามหลัก 3NF
- สัปดาห์ 10: เพิ่ม menu_items.image_url, employees.username / password_hash และ role owner, orders.queue_date / queue_no และตารางใหม่ branch_daily_counters

7. การออกแบบกระบวนการและพฤติกรรมระบบ

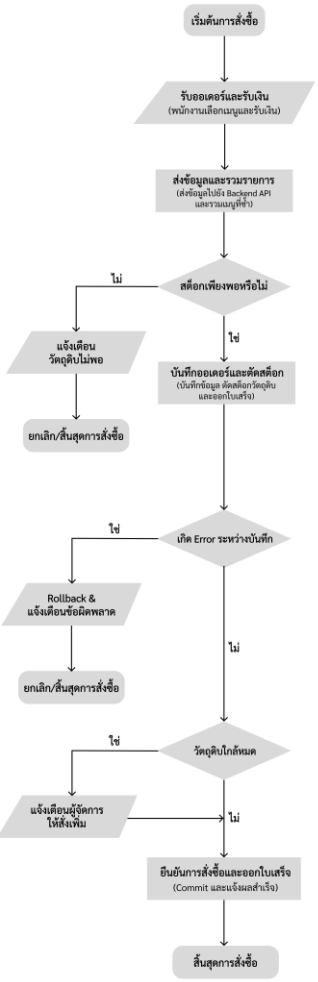
7.1 Sequence Diagram: สร้างออเดอร์ (POST /api/orders)



รูปที่ 7.1 Sequence Diagram การสร้างออเดอร์ (wk09)

7.2 Business Rules

Rule	ความหมาย	อยู่ในโค้ดที่
BR-01	ตรวจสต็อกวัตถุดิบก่อนยืนยัน ถ้าไม่พอตอบ 400 "วัตถุดิบไม่เพียงพอ"	orderController.createOrder
BR-02	รวมเมนูที่สั่งซ้ำเป็นรายการเดียว	orderController.createOrder
BR-03	บันทึก 5 ตาราง (ออเดอร์ รายการ สต็อก ประวัติสต็อก ใบเสร็จ) ใน transaction เดียว ถ้าพลาดยกเลิกทั้งหมด	orderController + orderModel
BR-04	แจ้งเตือนเมื่อสต็อกหลังตัดต่ำกว่าเกณฑ์	lowStockWarnings ใน createOrder



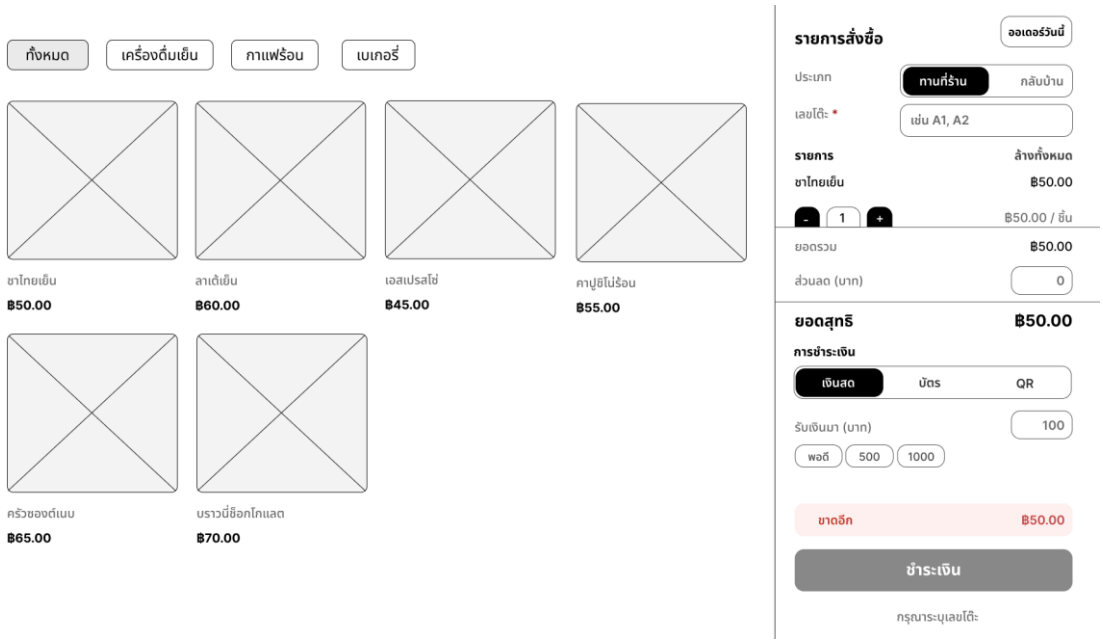
รูปที่ 7.2 Flowchart ของ Business Rule (wk09)

7.3 ความสอดคล้องกับโค้ด

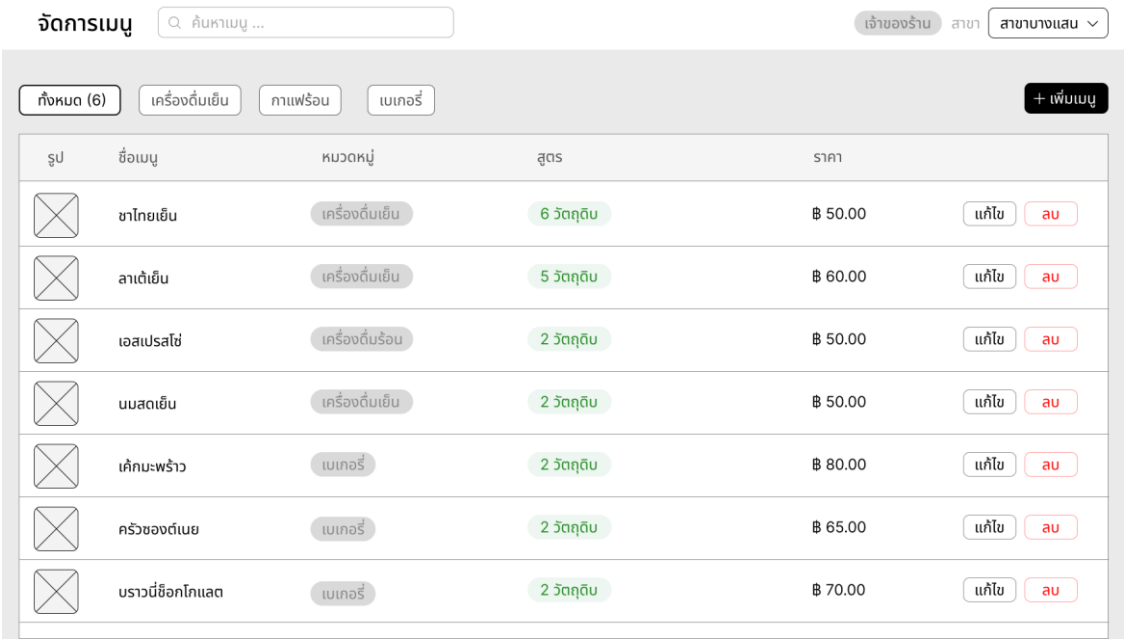
โค้ดจริงทำตามลำดับใน Sequence Diagram (รวมเมนูซ้ำ → ดึงสูตร → ตรวจสต็อก → transaction → ตอบ 201) และมีส่วนที่เพิ่มหลังออกแบบ ได้แก่ การออกเลขคิวรายวัน (nextQueueNo) และการตัดสต็อกแบบ "เช็กและตัดในคำสั่งเดียว" ที่เพิ่มในสัปดาห์ 14 เพื่อกันสต็อกติดลบเมื่อมีหลายออเดอร์พร้อมกัน (ดูหัวข้อ 12)

8. การออกแบบ Input/Output และ UI

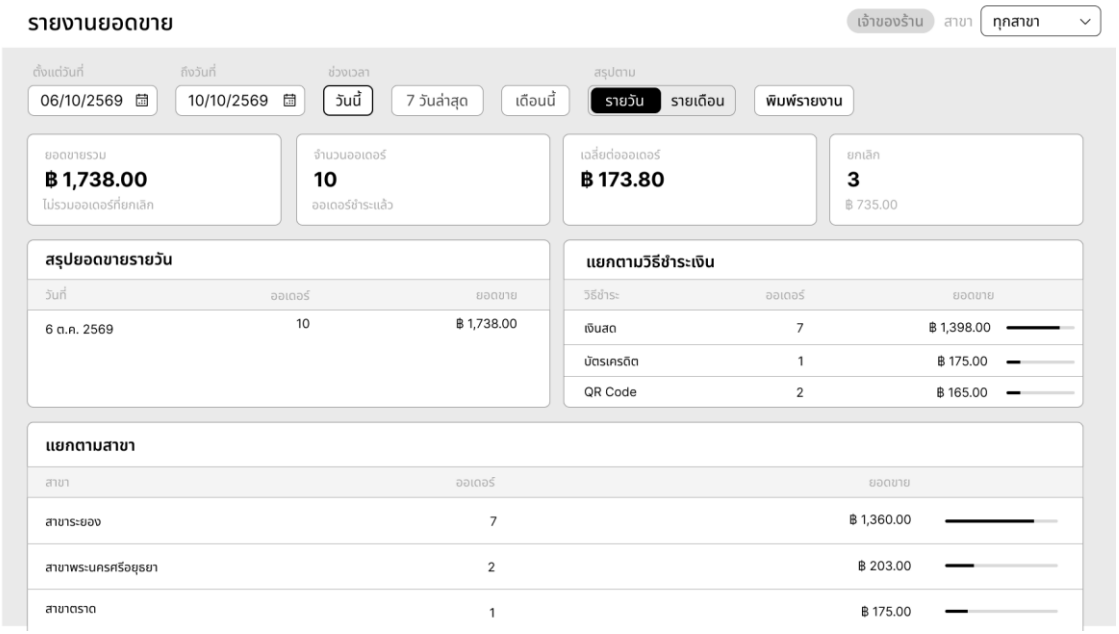
8.1 Wireframe (Mid-Fi)



รูปที่ 8.1 Wireframe หน้า POS รับออเดอร์

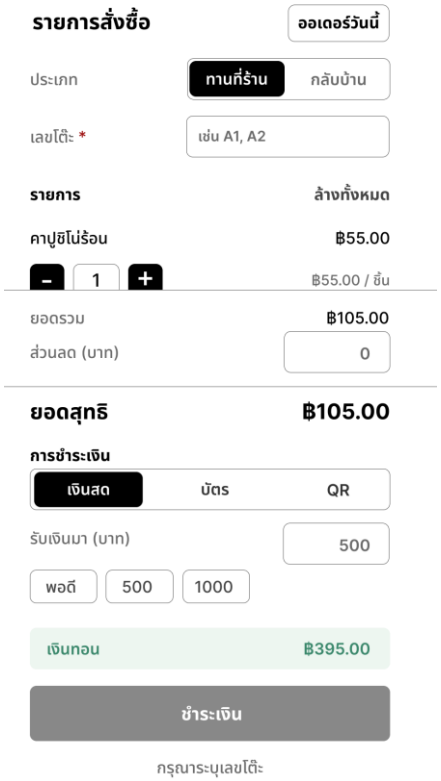


รูปที่ 8.2 Wireframe หน้าจัดการเมนู

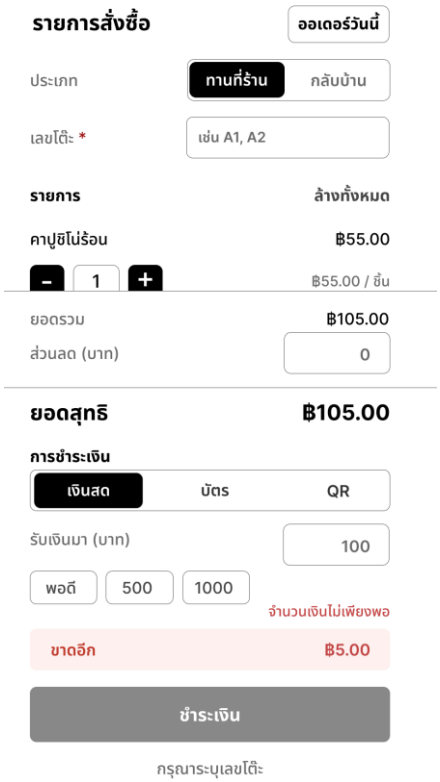


รูปที่ 8.3 Wireframe หน้ารายงานยอดขาย

8.2 หน้าจอที่ implement จริง



รูปที่ 8.4 หน้า POS รับออเดอร์ (เลือกเมนู ตะกร้า ส่วนลด วิธีชำระ)



รูปที่ 8.5 หน้า POS เมื่อเลือกกลับบ้านและชำระด้วยบัตร



Baristech Cafe

สาขาบางแสน
อ. เมือง จ. ชลบุรี

คิวที่

A006

เลขที่	REC-1791211756299-7
วันที่	5 ต.ค. 2569 21:49
แคชเชียร์	แคชเชียร์บางแสน
ประเภท	กลับบ้าน

ชาไทยเย็น	
1 × 50.00	50.00
ครัวซองต์เนย	
2 × 65.00	130.00
บราวนี่ช็อกโกแลต	
1 × 70.00	70.00

รวม	250.00
ส่วนลด	-25.00
ยอดสุทธิ	**225.00**
เงินสด	500.00
เงินทอน	275.00

ขอบคุณที่ใช้บริการ

รูปที่ 8.6 ใบเสร็จที่ระบบออกให้ (มีเลขคิว เลขที่ใบเสร็จ รายการ ยอดสุทธิ เงินทอน)

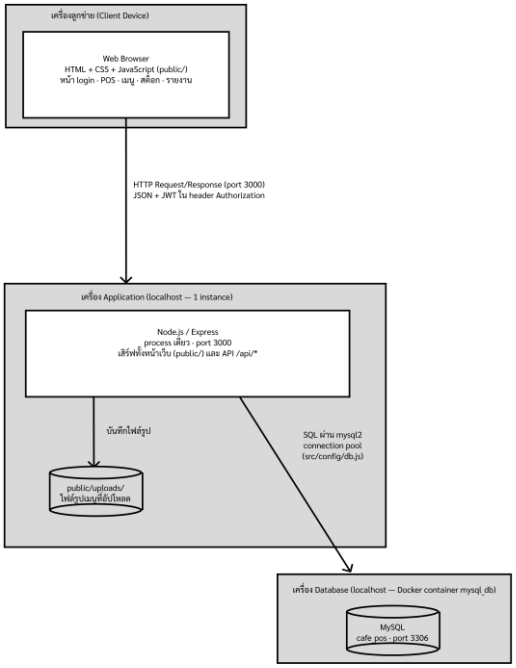
หน้าจอ	ผู้ใช้	ทำอะไรได้	สิ่งที่ตรวจทันทีที่ช่องกรอก
เข้าสู่ระบบ	ทุกคน	ล็อกอินแล้วเข้าหน้าตามบทบาท	กรอกครบหรือไม่
POS รับออเดอร์	แคชเชียร์	เลือกเมนู ตะกร้า ส่วนลด ชำระเงิน ออกใบเสร็จ	เงินไม่พอ・ส่วนลดเกินยอด・ทานที่ร้านต้องมีเลขโต๊ะ
ประวัติออเดอร์	แคชเชียร์	ดูออเดอร์วันนี้ พิมพ์ใบเสร็จซ้ำ ยกเลิกออเดอร์	ยกเลิกได้เฉพาะที่ยังไม่เริ่มทำ
จัดการเมนู	เจ้าของ	เพิ่ม/แก้ไข/ลบเมนู อัปโหลดรูป กำหนดสูตรวัตถุดิบ	ชื่อ ราคา > 0 สูตรไม่ซ้ำ
สต็อกวัตถุดิบ	เจ้าของ	เพิ่ม/แก้ไข/ลบ เติมสต็อก ดูรายการใกล้หมด	จำนวนไม่ติดลบ
แดชบอร์ดยอดขาย	เจ้าของ	สรุปยอดรายวัน/เดือน วิธีชำระ สาขา เมนู ดูใบเสร็จ ยกเลิกออเดอร์	ช่วงวันที่ถูกต้อง

9. สถาปัตยกรรมระบบ

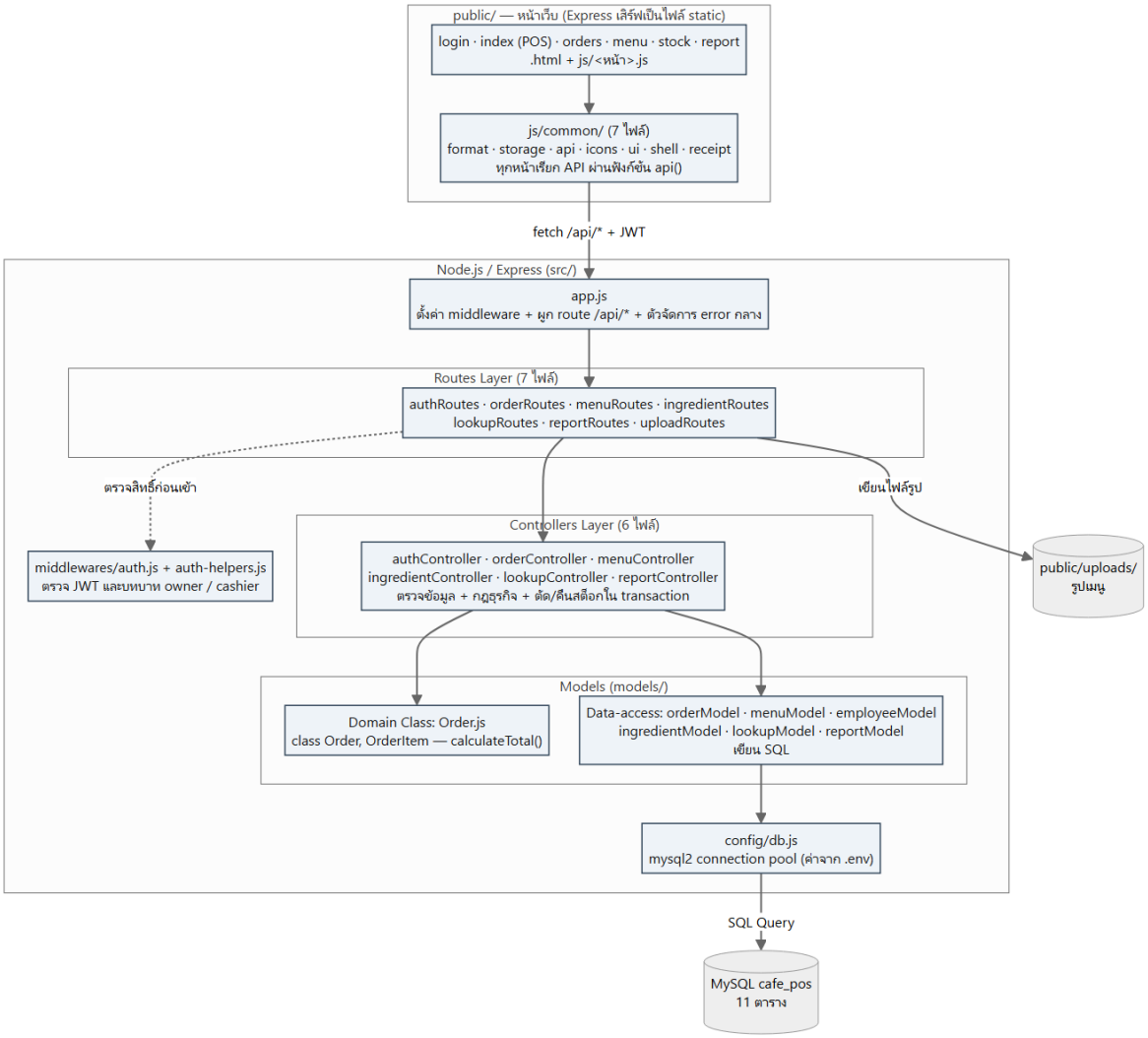
ระบบเป็น **Monolithic + Layered**: หน้าเว็บ API และการเข้าถึงฐานข้อมูลอยู่ใน codebase เดียว รันเป็น Node.js process เดียว (localhost:3000) ต่อกับ MySQL ใน Docker container บนเครื่องเดียวกัน ไม่แยกเป็น microservices เพราะทีมเล็ก เวลาจำกัด และขอบเขตระบบไม่ซับซ้อน ส่วน "หลายสาขา" แยกด้วย branch_id ในฐานข้อมูลเดียว

9.1 4+1 View

View	เอกสาร
Scenarios (+1)	Use Case Diagram (wk04)
Logical	Class Diagram (wk06)
Process	Sequence Diagram (wk09)
Physical	รูปที่ 9.1 (wk11)
Development	รูปที่ 9.2 (ฉบับปัจจุบัน)



รูปที่ 9.1 Physical View: เครื่องเดียว รัน Node.js และ MySQL (Docker)



รูปที่ 9.2 Development View ฉบับปัจจุบัน: routes → controllers → models → MySQL

9.2 การแบ่ง component

ชั้น	ไฟล์ใน repo (code/)	หน้าที่
Presentation	public/*.html, public/js/*.js, public/js/common/ (7 ไฟล์)	หน้าเว็บ และฟังก์ชัน api() ที่เรียก backend พร้อมแนบ token
Routes	src/routes/ 7 ไฟล์	กำหนด URL และตรวจสิทธิ์ตามบทบาท
Controllers	src/controllers/ 6 ไฟล์	ตรวจข้อมูล กฎธุรกิจ และ transaction
Models	src/models/ 7 ไฟล์ (รวม Order.js)	คำสั่ง SQL และคลาสธุรกิจ
Config / Middleware	config/db.js, middlewares/auth.js	เชื่อมฐานข้อมูล ตรวจ JWT

10. ผลการทดสอบ Usability

ทดสอบแบบ task-based พร้อม think-aloud กับผู้ทดสอบ 1 คน (เพื่อนต่างกลุ่ม) ที่ไม่เคยเห็นระบบ ทำ task บทบาทแคชเชียร์ที่เป็นเป้าหมายไม่ระบุขั้นตอน เช่น "ยอดรวม 150 บาท แต่กรอกเงินที่รับเป็น 100 บาท ระบบจะตอบสนองอย่างไร" (ยอดเงินไม่ครบ), "ลูกค้าสั่งครัวซองต์ที่หมด ระบบแสดงผลอย่างไร" (สต็อกไม่พอ), "ค้นหาบิลที่เพิ่งขายและพิมพ์ใบเสร็จซ้ำ" (ประวัติออเดอร์) ทั้งชุด task ฝั่งแคชเชียร์ 7 ข้อและเจ้าของร้าน 5 ข้อ ครอบคลุม happy path และ error path อยู่ใน wk12-usability-testing.md

#	ปัญหาที่พบ (คำพูดผู้ทดสอบ)	หลักการ	Severity	การแก้ไข (ทำแล้ว)
1	ช่องกรอกเงินสดรับมาสีอ่อนมาก กลืนไปกับหน้าจอ ต้องหาก่อนว่ากรอกตรงไหน	Visibility	Minor	ปรับขอบหนาสีเข้ม พื้นขาว ตัวหนา และปุ่มลัดเงินให้เด่นขึ้น
2	กดเพิ่มสินค้าได้เรื่อย ๆ ไม่รู้ว่าสต็อกเหลือเท่าไร ระบบเตือน "วัตถุดิบไม่พอ" ตอนกดชำระเงินแล้ว	Error Prevention, Feedback	Major	หน้า POS แสดง "เหลือ N" บนการ์ดเมนู เตือนทันทีและหยุดเพิ่มเมื่อถึงจำนวนที่ขายได้
3	เมนูประวัติ/รวมออเดอร์หายาก	Visibility	Major	เพิ่มหน้า "ประวัติออเดอร์" เป็นเมนูที่แถบซ้าย

11. ผลการรวมระบบ (Integration)

สัปดาห์ 13 รวมโค้ดทุกส่วนเป็น prototype เดียวครอบคลุม flow **สั่งออเดอร์ → ชำระเงิน → ใบเสร็จ → รายงานยอดขาย** เพิ่ม endpoint GET /api/reports/sales (สรุปยอดขายรวมต่อเมนู กรองตามสาขาและช่วงวันที่ นับเฉพาะออเดอร์ที่ชำระแล้ว เฉพาะเจ้าของ) แยก SQL ที่เคยอยู่ใน route ไปไว้ใน model และรวมโค้ดหน้าเว็บที่ใช้ร่วมกันไว้ใน public/js/common/

11.1 ผลทดสอบ End-to-End

ขั้นตอน	ผลที่คาดหวัง	ผล
ล็อกอินแคชเชียร์	เข้า POS ได้	ผ่าน
สั่งออเดอร์ + ชำระเงิน	ได้เลขคิว ตัดสต็อก ออกใบเสร็จ	ผ่าน
ล็อกอินเจ้าของ ดูรายงานยอดขายต่อเมนู	ยอดตรงกับออเดอร์ที่ชำระ ค่า totalSales เป็นตัวเลข	ผ่าน
ทดสอบข้ามสาขา	ออเดอร์สาขา B ไม่ปนในรายงานสาขา A	ผ่าน
แคชเชียร์เรียกรายงาน	ถูกปฏิเสธ (403)	ผ่าน
ทดสอบซ้ำทั้ง flow หลังปรับโค้ด	ผ่านครบ	ผ่าน

12. ผลการตรวจสอบคุณภาพและความปลอดภัย

ไล่ตรวจ **20 endpoint** ทุกเส้นเทียบ Checklist 5 ข้อ (Input Validation, IDOR, SQL Injection, Error Handling, Race Condition) ด้วยการอ่านโค้ดและรันทดสอบจริงบนฐานข้อมูลทดสอบชั่วคราว สคริปต์อยู่ที่ code/tests/quality-review/ ตารางผลรายละเอียดอยู่ใน wk14-quality-review.md

12.1 ความเสี่ยงที่พบ

#	ความเสี่ยง	ระดับ	สถานะ
1	สต็อกติดลบเมื่อมีออเดอร์พร้อมกัน (Race Condition ของ POST /api/orders) ทดสอบจริง: สต็อกเหลือ 1 ยิง 6 ออเดอร์พร้อมกัน รับครบ 6 ใบ สต็อกเป็น −5	High	แก้แล้ว
2	ฝั่ง server ไม่ตรวจเลขโต๊ะ (ทานที่ร้านไม่ใส่ก็รับ) และ amountReceived ที่ไม่ใช่ตัวเลข	Medium	แก้แล้ว
3	ส่ง JSON เสียแล้วระบบตอบหน้า HTML ที่มี stack trace	Medium	แก้แล้ว
4	ไม่ตรวจขนาดของข้อมูล (ชื่อยาว ตัวเลขใหญ่ วันที่ไม่จริง) ตอบ 500 แทน 400	Low	ยังไม่แก้
5	อัปโหลดรูปไม่ตรวจเนื้อไฟล์จริง	Low	ยังไม่แก้
6	ตัวนับล็อกอินในหน่วยความจำ, token ยกเลิกไม่ได้, token ใน localStorage	Low	ยังไม่แก้

12.2 การแก้ไขความเสี่ยงสูงสุด

แก้จาก "ดูสต็อกก่อนแล้วค่อยหัก" เป็น "ดูและหักในคำสั่ง SQL เดียว" (UPDATE ... WHERE stock_quantity >= ?) ถ้าสต็อกไม่พอจะยกเลิกทั้งออเดอร์และตอบ 400 ผลทดสอบซ้ำ: รับ 1 ใบ ปฏิเสธ 5 ใบ สต็อกเป็น 0 และมีออเดอร์ถูกบันทึกเพียง 1 ใบ

12.3 ผลทดสอบหลังแก้

- Acceptance Criteria สัปดาห์ 4 (US-01 ถึง US-06): **ผ่าน 18/18 ข้อ** (รวมยกเลิกออเดอร์แล้วคืนสต็อก)
- IDOR ข้ามสาขา, SQL Injection, Error Handling: **ผ่าน 20/20 ข้อ** (เช่น แคชเชียร์สาขา 1 เปิดออเดอร์สาขา 2 ได้ 404, ชื่อเมนูที่มี DROP TABLE ไม่เกิดผล)

ความเสี่ยงระดับ Low (#4–#6) มีแผนแก้ไขในรายงานสัปดาห์ 14 ไม่ได้ปล่อยผ่านโดยไม่บันทึก

13. สรุปผลและข้อเสนอแนะ

13.1 สิ่งที่ทำได้

- ระบบทำงานครบ flow สั่งออเดอร์ → ชำระเงิน (เงินสด บัตร QR) → ใบเสร็จ → รายงานยอดขาย หลายสาขา แยกสิทธิ์เจ้าของ/แคชเชียร์ที่ฝั่ง server
- ตัดสต็อกอัตโนมัติตามสูตร มีประวัติ ยกเลิกออเดอร์แล้วคืนสต็อก และเลขคิวรายวันแยกตามสาขา
- ผ่านการทดสอบ usability (แก้ครบ 3 ข้อ) และการตรวจคุณภาพ/ความปลอดภัย (แก้ความเสี่ยง High และ Medium ทั้งหมด)

13.2 บทเรียน

- การออกแบบที่ต่อเนื่องกันทุกสัปดาห์ (BPMN → Requirement → User Story → Diagram → โค้ด) ช่วยให้ทวนกลับหาที่มาของแต่ละส่วนของระบบได้
- diagram กับโค้ดจริงเริ่มต่างกันได้ตลอด (เช่น สต็อกย้ายจากเมนูไปอยู่ที่วัตถุดิบ, เพิ่มเลขคิว) ต้องกลับมาปรับเอกสารเมื่อโค้ดเปลี่ยน
- การทดสอบกับผู้ใช้จริงและการไล่ตรวจตาม checklist เจอปัญหาที่ทีมมองไม่เห็นเอง เช่น การเตือนสต็อกช้าเกินไป และสต็อกติดลบเมื่อกดชำระเงินพร้อมกัน

13.3 ข้อจำกัดของระบบ

- ยังไม่มีหน้าจอฝั่งบาริสต้า (FR-03) สถานะ "กำลังทำ" ต้องแก้ในฐานข้อมูลเอง
- วัตถุดิบใช้ร่วมกันทุกสาขา (สต็อกก้อนเดียว) ตาม schema; จำนวน "เหลือ N" บนหน้า POS คำนวณทีละเมนูจึงเป็นแค่การเตือนล่วงหน้า (server ตรวจซ้ำตอนชำระเงินเสมอ)
- ความเสี่ยงระดับ Low ในหัวข้อ 12 ยังไม่ได้แก้ และผลทดสอบ usability มาจากผู้ทดสอบ 1 คน

13.4 แนวทางพัฒนาต่อ

- ทำหน้าจอบาริสต้า (FR-03) และแยกสต็อกรายสาขา
- แก้ความเสี่ยงระดับ Low: ตรวจขนาดข้อมูล ตรวจเนื้อไฟล์รูป เก็บ token แบบ httpOnly cookie และเพิกถอน token ได้
- ให้ผู้ใช้จริงหลายคนทดสอบซ้ำหลังแก้ usability และทดสอบภาระสูง

ภาคผนวก

ภาคผนวก ก: AI Disclosure Form ที่ใช้ตลอดเทอม

แบบฟอร์มฉบับเต็มทุกสัปดาห์อยู่ในโฟลเดอร์ docs/ai-disclosure/ สรุปได้ดังนี้

สัปดาห์	เครื่องมือ AI	งานที่ใช้ AI	ไฟล์
5	ChatGPT, Gemini	ช่วยเขียน/ปรับ orderController (System Architecture)	wk05-ai-disclosure.md
7	ChatGPT	สอบทาน Use Case เทียบ ER Diagram, ออกแบบ ER/Entity/Cardinality, ตรวจ 3NF, schema.sql และ Model	wk07-ai-disclosure.md
9	ChatGPT	วิเคราะห์ Business Rules, ปรับ schema (branch_id, voided, ตัดค่าที่คำนวณได้), Sequence Diagram/Flowchart, API เมนูและออเดอร์ด้วย transaction	wk09-ai-disclosure.md
10	Claude Code	หน้าเว็บ POS/เมนู/สต็อก/รายงาน/ล็อกอิน, API เพิ่มเติม, JWT และแยกสิทธิ์, เอกสาร I/O design	wk10-ai-disclosure.md
12	ChatGPT, Gemini	ร่าง task สำหรับ usability testing (นิสิตเป็นผู้ทดสอบและประเมินเอง)	ท้ายไฟล์ wk12-usability-testing.md
13	Claude Code	endpoint รายงาน, แยก route/controller/model, แยก common/, แก้ usability, ปรับ requirement, เอกสารอธิบายโค้ด	wk13-ai-disclosure.md

ภาคผนวก ข: เอกสารที่เก็บแยกไฟล์

- SRS ฉบับสมบูรณ์: docs/wk10-srs.md (รวมความต้องการ wk03–04, ER wk07 และ Input/Output wk10)
- Input/Output Design และ Wireframe ฉบับเต็ม: docs/wk10-input-output-design.docx, docs/wk10-wireframe.docx
- เอกสารอธิบายโค้ดตามภาพหน้าจอ: docs/code-guide/